Admin By Request Intune packaging: Win32 app to production

Technical deep dive: wrap the Admin By Request Windows client with the Win32 Content Prep Tool (IntuneWinAppUtil.exe) and ship it as an Intune Win32 app. Folder layout, exact packaging command, install and version-independent uninstall commands, file-version detection, requirements, return codes, pilot-to-wave assignments, verification, Intune supersedence vs vendor Auto-Update, IME logs, error codes and rollback.

9 phases | 24 steps | Tick each item when its "done when" criterion is true

Phase 1 Prerequisites: devices, roles and the version-ownership decision about Half a day

- [ ] **1.1 Confirm the devices can take a Win32 app**
  Every target device is Entra joined, hybrid joined or Entra registered, enrolled in Intune, runs a supported Pro, Enterprise or Education edition, and its user holds an Intune licence
- [ ] **1.2 Give the packager least-privilege Intune rights**
  The account that creates the app holds the Intune built-in role Application Manager (or a custom role with Win32 app and supersedence rights), not Global Administrator, and uses PIM where available
- [ ] **1.3 Decide who owns client versions before you package**
  One written decision per platform: either Intune owns versions (Auto-Update for Windows Workstation off) or Auto-Update owns them (Intune app used for new installs only)

Phase 2 Get the Content Prep Tool and lay out the packaging folders about 30 minutes

- [ ] **2.1 Download and verify IntuneWinAppUtil.exe**
  `IntuneWinAppUtil.exe -v` on the packaging machine prints the current release from the GitHub repository (1.8.7 at the time of writing)
- [ ] **2.2 Separate tool, source and output folders**
  The tool sits outside every source folder, each source folder contains only the Admin By Request MSI, and the output folder is not inside the source folder

Phase 3 Package the MSI into an .intunewin file about 30 minutes

- [ ] **3.1 Download the MSI and record its full four-part version**
  The MSI from the portal's Download menu is in the version's source folder, and its four-part version (for example `8.5.1.0`) is recorded in the change ticket
- [ ] **3.2 Run IntuneWinAppUtil.exe with exact, quoted paths**
  The output folder contains a `.intunewin` named after the MSI, and the tool reported no errors
- [ ] **3.3 Inspect the package and store it like the installer**
  A copy of the .intunewin opened as a ZIP shows a `Contents` and a `Metadata` folder, and the file is stored with restricted access next to the MSI

Phase 4 Create the Win32 app: information, program and requirements about 1 hour

- [ ] **4.1 Upload the package and fill in app information**
  The app exists under `Apps > Windows` as a Windows app (Win32) with a unique name, publisher, and the full four-part version as app version
- [ ] **4.2 Set install and uninstall commands, context and restart behaviour**
  Install command is `msiexec /i "<exact file name>.msi" /qn`, the uninstall command is the vendor's `Get-Package` PowerShell line, install behaviour is System, and `Allow available uninstall` is No
- [ ] **4.3 Set requirements: architecture and minimum OS**
  Requirements match the vendor's guidance (x64, minimum Windows 10 1607) or a stricter floor that reflects your supported Windows versions, and Arm devices are handled deliberately

Phase 5 Detection rule, dependencies and supersedence about 30 minutes

- [ ] **5.1 Detect AdminByRequest.exe by file version, never by MSI product code**
  One manual detection rule of type File: path `C:\Program Files (x86)\FastTrack Software\Admin By Request`, file `AdminByRequest.exe`, detection method String (version), operator Greater than or equal to, value the full four-part version

- [ ] **5.2 Leave dependencies and supersedence empty for the first version**
  The first Admin By Request app has no dependencies and supersedes nothing, and a reviewer has checked every tab on Review + create

Phase 6 Assign: pilot device group, then waves about 1–3 weeks

- [ ] **6.1 Assign Required to a pilot device group**
  The app is assigned with the Required intent to a pilot device group of 20–50 devices spanning all personas, and nothing else is assigned yet
- [ ] **6.2 Tune notifications, restart grace and delivery**
  End-user notifications, availability, deadline and (if used) the restart grace period are set on each assignment, and Delivery Optimization priority is chosen
- [ ] **6.3 Roll out in waves by device group**
  Each wave is a separate device group added to the same app, with a go/no-go based on install success, Not applicable count and elevation tickets from the previous wave

Phase 7 Verify the install end to end about 1 hour per wave

- [ ] **7.1 Check Intune status per device, not on the overview**
  For the sample devices, `Devices > <device> > Managed Apps` shows Admin By Request Workstation as Installed
- [ ] **7.2 Check the agent on the endpoint and in the portal**
  The program folder exists, `AdminByRequest.exe` runs, the MSI Installer events are in the Application log, Connectivity shows OK, and the device is in the portal inventory
- [ ] **7.3 Prove elevation works and is audited**
  A pilot user has completed one Run As Admin (and, if allowed, one Admin Session) and both appear in the portal audit log with the expected sub-setting

Phase 8 Update the client: supersedence or Auto-Update about Per release

- [ ] **8.1 If Intune owns versions: package the new release and supersede**
  For each release there is a new app with its own .intunewin, detection value raised to the new four-part version, a supersedence entry on the old app with `Uninstall previous version` set to No, and its own explicit assignment
- [ ] **8.2 If Auto-Update owns versions: keep the Intune app for new installs**
  Auto-Update for Windows Workstation is on, the Intune app's detection rule is still `Greater than or equal to` an older version, and the app is repackaged at major releases only

Phase 9 Troubleshooting and rollback about Ongoing

- [ ] **9.1 Read the IME logs and collect them from Intune**
  The runbook names the log folder, the three logs to open first, and how to collect them remotely
- [ ] **9.2 Map the common symptoms to their cause**
  Each symptom below has a known fix in the runbook, and the team has seen at least one of them in the pilot
- [ ] **9.3 Roll back: smallest lever first, uninstall intent last**
  The team can, in order: relax the portal sub-setting, stop a wave, and uninstall from a group with the Uninstall intent, and each device has a documented admin recovery path

Owner	Change / ticket	Date completed

Original checklist by Tenant Recipes, an independent project not affiliated with Microsoft or any vendor. Every step, its actions, pitfalls and the official sources it was checked against are on https://tenantrecipes.com/guides/admin-by-request-intune-win32-packaging.html. Time estimates are our own planning figures. Confirm against the linked sources and test in a pilot before production.